Bibliography

[1] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus, “Fine-grained and accurate source code differencing.”

[2] Dyer, Robert, Rajan, Hridesh, Nguyen, Hoan Anh, and Nguyen, Tien N., “Mining billions of AST nodes to study actual and potential usage of Java language features,” *Association for Computing Machinery*.

[3] Lämmel, Ralf, Pek, Ekaterina, and Starek, Jürgen, “Large-scale, AST-based API-usage analysis of open-source Java projects.”

[4] Arafat, O. and Riehle, D., “The Commit Size Distribution of Open Source Software.”

Things to know when building tools that work on code

Why is this not common knowledge?

MARKO IVANKOVIĆ[1]

[1]marko@ivankovic.me, Universität Passau

BMW
München, 09-10.3.2026.

Overview

1. Background
2. Existing research
3. Let's dig
4. Rules of thumb
5. What am I doing next?

BACKGROUND

Background

- As an industry, we make a lot of tools for Software Engineers.
- Many of these tools work on code.
- But do we know how code actually looks like?

Why does this matter?

- How often did you see the “worst case” “upper bound” Big-O notation in papers?
- As an example, in “Fine-grained and Accurate Source Code Differencing” [1], the authors state that the complexity of their algorithm is O (n^2) where n is the maximum of the number of nodes in the two trees being compared.
- What is the maximum number of nodes?
  - What is the theorethical maximum number of nodes?
  - What is the physically possible maximum number of nodes?
  - What is the actual maximum number of nodes on the Planet Earth today?
  - What is the maximal number of nodes your tool will ever see?
  - What is the second largest number? Or how about 99.99th percentile?

Difference between engineering and science

- If civil engineers built houses the way we build software, every house would be designed to survive a volcanic eruption happening at the same time as a nuclear blast… underwater becase a tsunami just passed over the area.
- When you build a house, the first step is to get a geological engineer to do ground analysis and then you design your house based on actual constraints, not based on possible constraints.
- Very often, we build the tool first, using theorethical “worst case” analysis to guide the developement, and then we measure the actual performance after the tool is built.

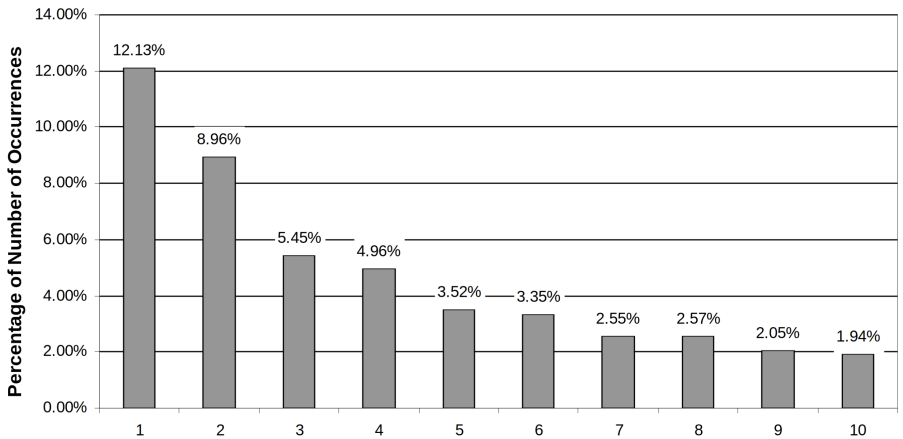
EXISTING RESEARCH

Two separate papers that use AST nodes to look for API usage

- “Mining Billions of AST Nodes to Study Actual and Potential Usage of Java Language Features” [2] and “Large-scale, AST-based API-usage analysis of open-source Java projects” [3] both use AST analysis to find function calls and analyze API usage
- Neither provides a comprehensive analysis of size, except for offhand sentences like “In this paper, we analyze over 31k opensource Java projects representing over 9 million Java files, which when parsed contain over 18 billion AST nodes”
- About 2 thousand nodes per file on average then :)
- Java only

The Commit Size Distribution of Open Source Software [4].

- Best paper I found so far for commit sizes
- 8M commits across 9363 projects
- Key-takeaway: Distribution of commit sizes is logarithmic.
  - 47.5% of all commits are 10 or fewer lines.
  - Commits beyond 10000 lines do exist.
- Some limitations:
  - Doesn't provide a per-language overview
  - LOC focus, not AST nodes



LET’S DIG

Let's find the answer to our questions

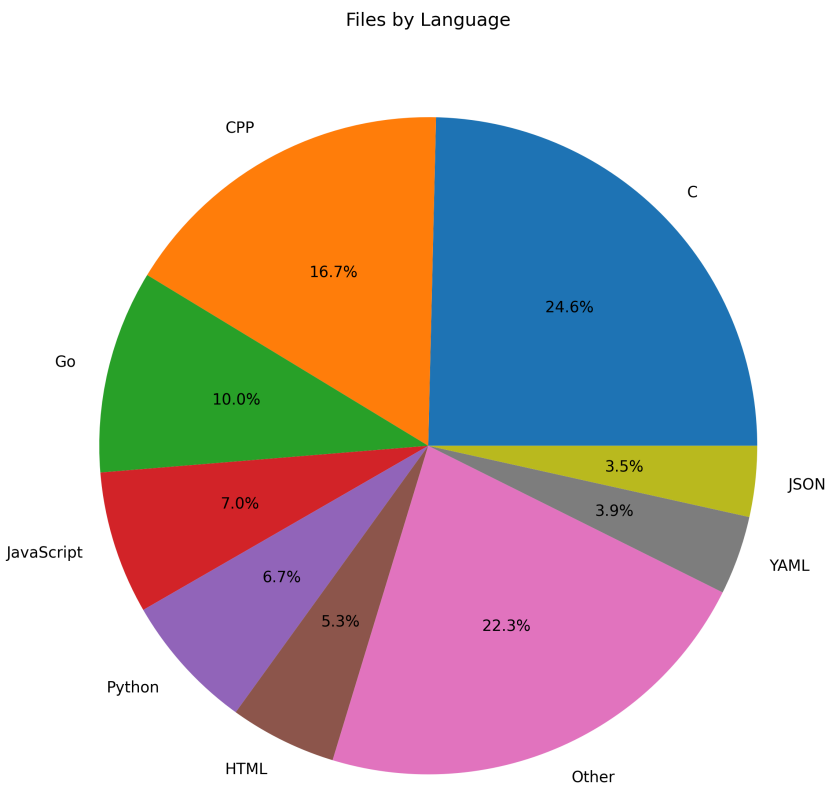
- What is the theorethical maximum number of nodes?
  - Ok this is probably ∞.
- What is the physically possible maximum number of nodes?
  - There are 10^{80} atoms in the universe. So if we converted the entire universe into RAM… A lot, but infinitely fewer than ∞.
- What is the actual maximum number of nodes on the Planet Earth today?
- What is the maximal number of nodes your tool will ever see?
- What is the second largest number? Or how about 99.999th percentile?

What is the actual maximum number of nodes on the Planet Earth today?

- We can't know this. Too many private repositories. But we can make a decent educated guess.
- Let's take the top 1000 open source repositories, let's add the roughly 7300 more repositories that make up the Gentoo Linux distribution.
- Let's clone them all and count.

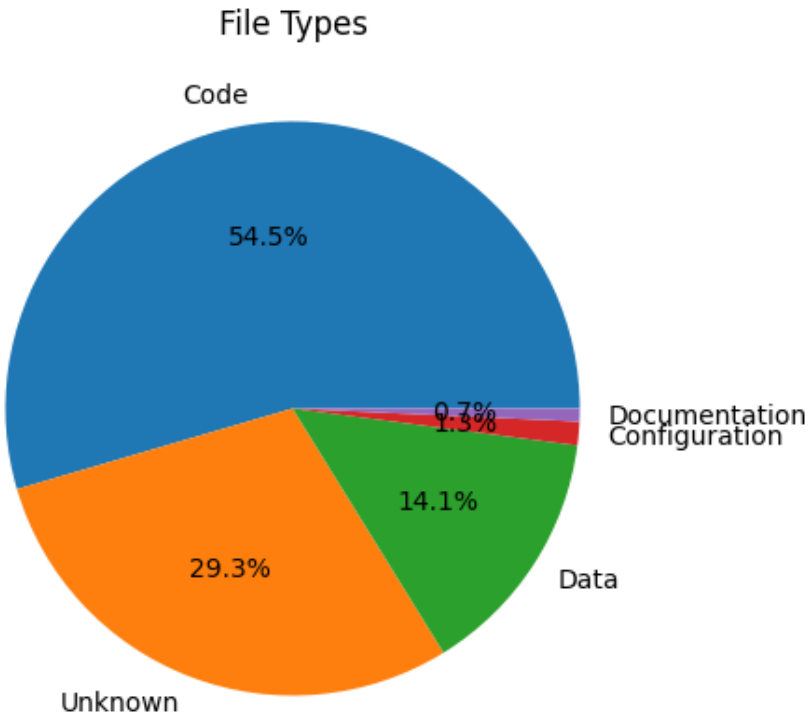
Attack of the clones

- Repositories: 7423
- Number of files: 6169828
- Languages: 31
- 4 types of files



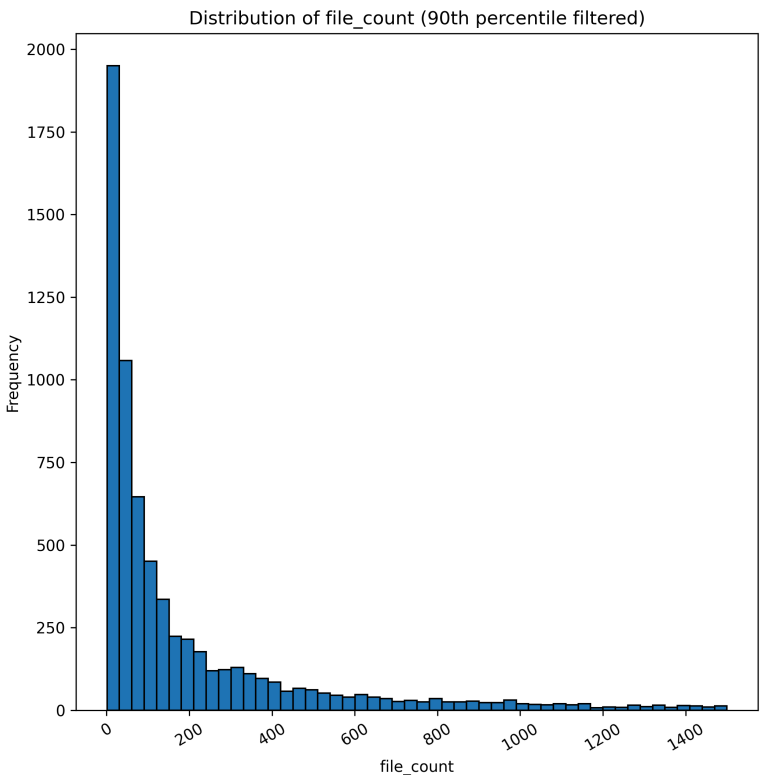
Repositories contain a lot of stuff, not just code

- Code
  - This is what you expect to find
- Configuration
  - Usually human readable. Yaml and similar formats
  - A lot of it is configuring the developement environment
- Data
  - Images, audio, video…
  - Test data
  - Data stored as very long arrays directly in code
- Documentation
  - Project guides, e.g. README.md
  - Legal, e.g. licences



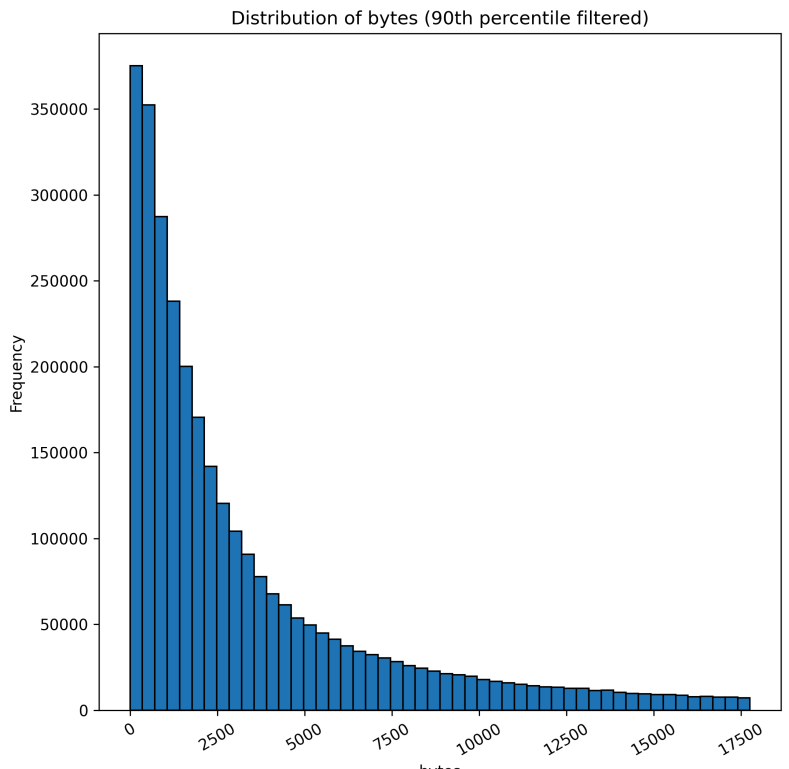
Files per project

- Median: 93
- 90th percentile: 1 500
- 99th percentile: 11 769
- 99.9th percentile: 49 049



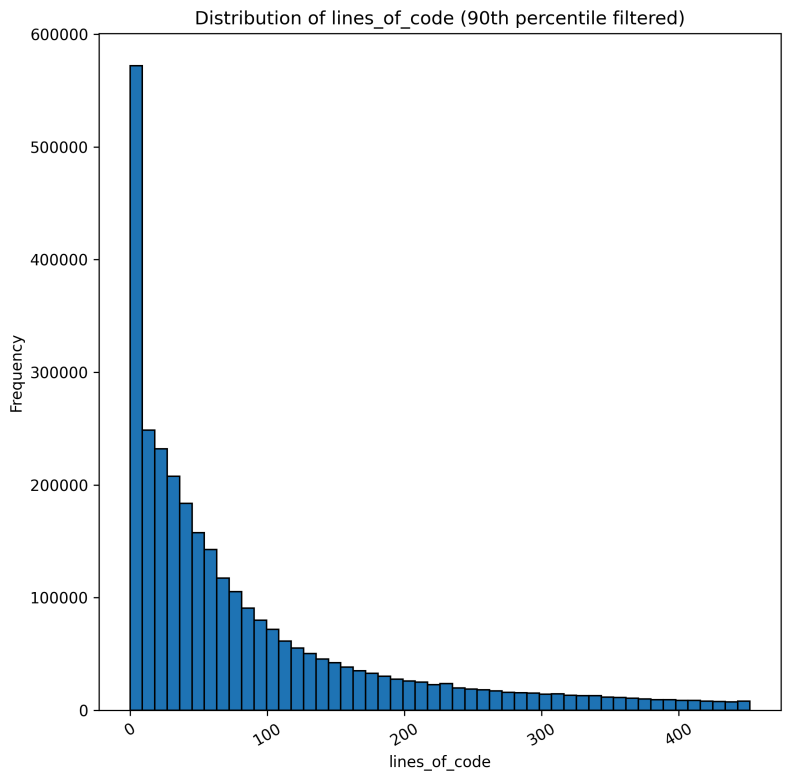
Bytes per file (Code files only)

- Median: 2 268
- 90th percentile: 17 759
- 99th percentile: 134 118
- 99.9th percentile: 1 193 619



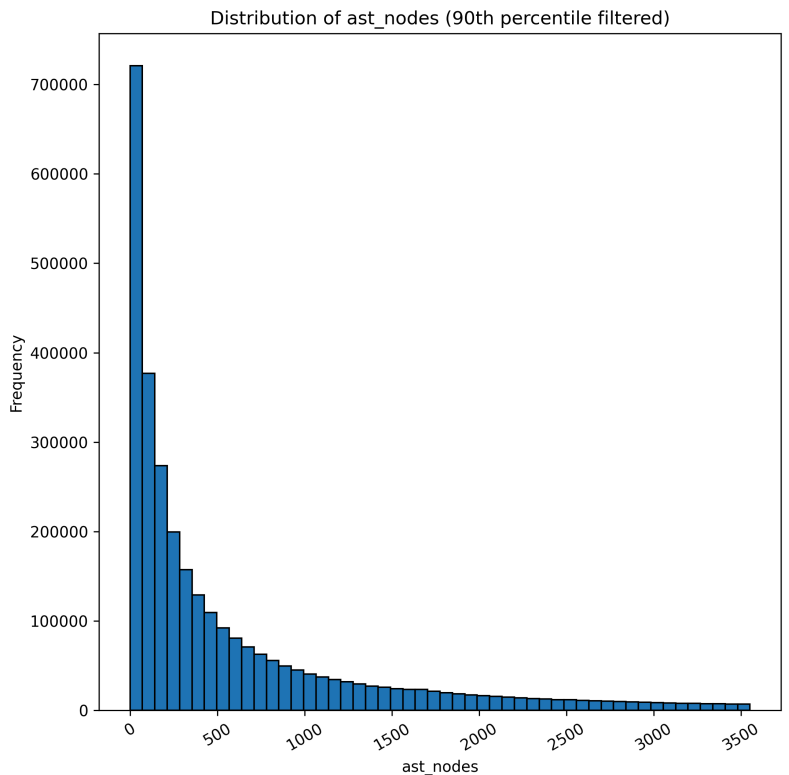
LOC per file (Code files only)

- Median: 59
- 90th percentile: 452
- 99th percentile: 2 780
- 99.9th percentile: 12 909



AST nodes per file (Code files only)

- Median: 332
- 90th percentile: 3 552
- 99th percentile: 24 516
- 99.9th percentile: 138 995
- 99.99th percentile: 289 352
- Maximum: 905 004



Correlation between bytes and AST nodes (Code files only)

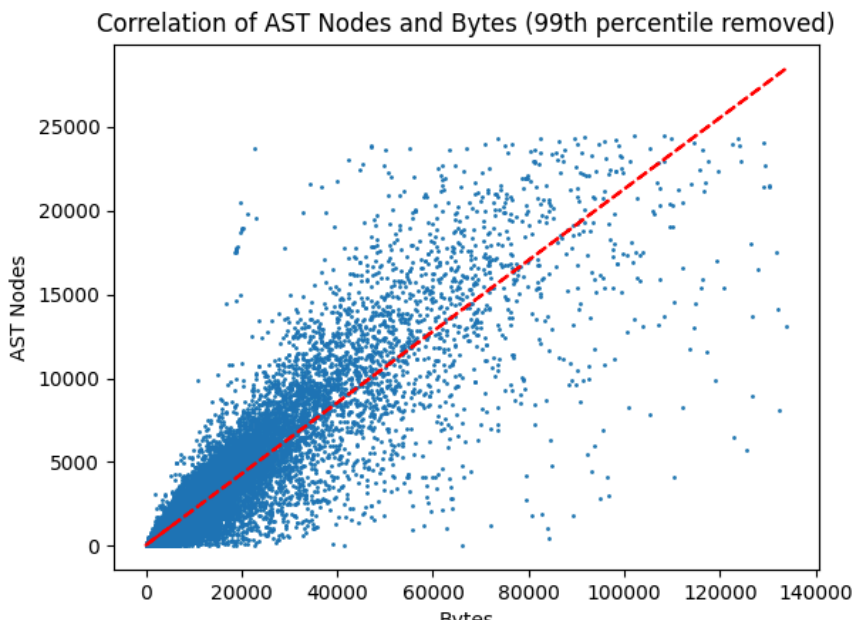
- Pearson correlation: 0.8952
- Formula:

$$|\text{AST}| = \text{bytes} * 0.2124 + 49.9386$$

- More useful formula:

$$|\text{AST}| = \frac{\text{bytes}}{5}$$

- This allows comparing algorithms that operate on bytes and AST nodes!
  - E.g. $O(\text{bytes}^2) = O(|\text{AST}|^2)$



Differences between selected languages - Bytes

Language	p50	p90 ↓	p99	p99.9	p99.99	Max
JavaScript	1019.0	7500.0	78635.0	732410.0	3328460.0	13049704
Java	2504.0	12659.0	61808.0	258290.0	2388576.0	4415767
TypeScript	1360.0	16440.0	435889.0	1671781.0	2083552.0	13049673
Go	3022.0	17526.0	103888.0	663969.0	2421009.0	19056507
Rust	1582.0	18283.0	97610.0	327338.0	2090774.0	8121838
Python	2699.0	19026.0	88459.0	324298.0	1164657.0	9570626
C	3364.0	23671.0	146836.0	1120632.0	8745894.0	83683072
CPP	3564.0	23857.0	123107.0	623365.0	4716925.0	33178839

Differences between selected languages - Lines

Language	p50	p90 ↓	p99	p99.9	p99.99	Max
JavaScript	30.0	188.0	1347.0	9036.0	20625.0	65564
Java	70.0	341.0	1525.0	4525.0	16677.0	40711
Go	67.0	412.0	1658.0	4213.0	10170.0	30613
TypeScript	43.0	456.0	7917.0	17819.0	22567.0	35452
Python	78.0	504.0	2187.0	6289.0	14392.0	24187
Rust	53.0	517.0	2364.0	7102.0	18398.0	33273
CPP	104.0	649.0	2973.0	9768.0	24362.0	196613
C	98.0	690.0	3560.0	12148.0	22979.0	41797

Differences between selected languages - AST Nodes

Language	p50	p90 ↓	p99	p99.9	p99.99	Max
JavaScript	175.0	1785.0	18854.0	143461.0	446488.0	619668
Java	379.0	2433.0	11496.0	33940.0	144123.0	266389
Go	443.0	3353.0	14408.0	40967.0	115607.0	506805
Python	545.0	4035.0	18162.0	59229.0	188399.0	333352
TypeScript	310.0	4156.0	110088.0	230432.0	310698.0	391335
Rust	405.0	4416.0	20070.0	59590.0	174416.0	266154
C	490.0	4714.0	27478.0	107231.0	259864.0	594136
CPP	692.0	5650.0	28229.0	100524.0	239858.0	594193

RULES OF THUMB

Quick rules of thumb when time is precious

- Repository composition: Two-thirds code, one third other stuff.
- Files per project: Tens of thousands mostly, can be hundreds of thousands.
- Bytes per file: Tens of thousands mostly, can be half a million. Million is rare.
- AST nodes per file: A fift of the number of bytes. Thousands to tens of thousands. Quarter million is rare.
- Commit size: Half of all commits is under 10 lines. But they go up to tens of thousands of lines.

WHAT AM I DOING NEXT?

CodeDiff

- I really hate how code diffing works today. If you have ever seen a GitHub diff where somebody inlines Python code in a new function, you know what annoys me.
- Theorethical algorithms are $O(|AST|^2)$. And some have been implemented, but in their Documentation they explicitly say they are not aiming to be robust and fast for production usage. They are mostly research artefacts.
- The difference between:
  - Make an algoritmically better code diffing algorithm faster than $O(|AST|^2)$
  - vs
  - Make a diffing binary that can diff up to 1M AST nodes, with 47.5% of all diffs being 10 or fewer lines. The diff must be computed under 100ms for 99.99% of all commits

It is easier to make a robust tool if you know the real constraints.

Thank you for your attention

marko@ivankovic.me